Super Admin Performance Audit — Slow Actions + Non-Realtime UI

Date: 2026-10-04 **Scope:** `src/app/super-admin/**` — action latency ("processing too long") and list freshness ("data not instant without refresh") **Standard:** Fast read/write path for dashboards — optimistic UI + silent revalidate, server pagination, parallel writes, scoped realtime, layered cache invalidation **Repo:** `chikwaaa212/pgso`, branch `main`

---

1. Executive Summary

Super Admin feels slow for two independent reasons that stack:

1. **Mutations block the UI on a long serial server round-trip.** Every button (`Approve`, `Deactivate`, `Complete`, etc.) does `setBusy(true) → await serverAction() → reload()`. The server action itself awaits auth → DB read → DB write → audit-log write → Redis SCAN/DEL bust → `revalidatePath`, all sequentially, before the client even starts its refetch. No optimistic update.
2. **Reads are double-cached with no realtime on most pages.** Server Redis TTL (30–60s) sits under client `useCachedAction` TTL (30–60s, `maxAge` 10 min + `sessionStorage`). Supabase Realtime (`useAdminRealtime`) is wired on only **3 of ~14 list pages** and silently no-ops on RLS/publication failure with no polling fallback. `revalidatePath()` does not clear the browser `useCachedAction` copy. Personnel writes bust only `personnel:*`, never `super-admin:*`.

   Result: user clicks → spinner for seconds → toast → list still shows old data until TTL expires or manual refresh.

---

2. Architecture (as-found)

```
Client page ("use client")
  └ useCachedAction(KEY, serverAction, { staleTime })   ← memory + sessionStorage, SWR-style
      └ Server Action ('use server' in browse/actions.ts, requests/actions.ts, users/actions.ts, ..
          ├ allowed() / requireSuperAdmin()           ← getPersonnelScope / profile lookup
          ├ withScopedCache('super-admin:<list>', 30|60) ← Upstash Redis read-through
          │    └ Prisma findMany({ take: 500 }) + profile lookups + (assets: per-row QR)
          └ mutation: Prisma write → writeAuditLog → bustSuperAdminScopes → revalidatePath

Realtime (opt-in per page):
  └ useAdminRealtime({ channel, tables, onEvent: refresh }) ← Supabase postgres_changes, 2s coalesce
```

Key files:

Concern	File
Client SWR cache	`src/hooks/use-cached-action.ts`, `src/lib/client-cache.ts`
Server Redis cache + bust	`src/lib/personnel-cache.ts`, `src/lib/redis.ts`
Realtime hook	`src/hooks/use-admin-realtime.ts`
Read layer	`src/app/super-admin/browse/actions.ts`
Mutations	`src/app/super-admin/requests/actions.ts`, `src/app/super-admin/users/actions.ts`, `src/app/super-admin/dashboard/actions.ts`, `src/app/super-admin/master-data/actions.ts`, `src/app/super-admin/inventory/actions.ts`
Representative pages	`src/app/super-admin/requests/page.tsx`, `src/app/super-admin/assets/page.tsx`, `src/app/super-admin/users/page.tsx` + `user-actions.tsx`, `src/app/super-admin/dashboard/page.tsx` + `stats.ts` + `analytics.ts`, `src/app/super-admin/deliveries/page.tsx`
Auth/scope	`src/lib/auth-guard.ts`, `src/lib/personnel-scope.ts`, `middleware.ts`

---

3. Finding A — Why "processing the action is too long"

A1. No optimistic UI — button waits for full round-trip + refetch [HIGH]

`src/app/super-admin/requests/page.tsx:92-119`:

```
setSaving(true);
const res = await setReplenishmentStatus(actionRow.id, actionStatus, note);
// ... only now:
setDialogOpen(false);
toast.success(successTitle);
reload(); // → refresh() → refetch 500-row list + 5000-row audit scan
```

`src/app/super-admin/users/user-actions.tsx:30-47`: same pattern (`setBusy(true) → await setUserActive() → onSuccess()`).

The spinner (`Approving… / Deactivating…` via `ActionButton loadingLabel`) covers the **entire** server chain **plus** the follow-up list refetch. Standard fast approach is the opposite: patch the row locally in <50 ms, send the write in background, rollback only on error.

A2. Server action awaits serial slow work before returning [HIGH]

`src/app/super-admin/requests/actions.ts:34-84`:

```
const session = await requireSuperAdmin();          // 1 round-trip
const current = await prisma.request.findUnique(...); // 2
await prisma.request.update(...);                    // 3
await writeAuditLog({...});                          // 4 (separate DB write)
await bustSuperAdminScopes(['requests']);            // 5 (Redis SCAN count:100 + DEL, 5s budget)
revalidatePath('/super-admin/requests');             // 6
revalidatePath('/personnel/requests');               // 7
return { success: true };
```

`src/lib/personnel-cache.ts:12-15,33-46`: `BUST_TIMEOUT_MS=5000`, `SCAN count:100`, up to `BUST_MAX_SCAN_PAGES=20`. `CACHE_IO_TIMEOUT_MS=2500` per Redis op. Audit logging and cache busting are on the critical path but do not need to block the success response — they can run in `after()` / fire-and-forget with correct ordering.

Same shape in `dashboard/actions.ts` (Supabase `auth.admin.createUser` + `prisma.profile.upsert` + `writeAuditLog` + idempotency DB trips + bust + 2× `revalidatePath`) and `users/actions.ts` (`listUsers(perPage:100)` + page 2 fetch inside the toggle path).

A3. Lists fetch full tables, paginate in the browser [HIGH]

Reader	Query	UI
`browseDeliveries` (`browse/actions.ts:89-93`)	`prisma.delivery.findMany({ take: 500, include: { _count } })` + `profile.findMany`	`BrowseTable` `slice(page)` shows 10
`browseRequests` (`browse/actions.ts:328` → `getRequestsPage pageSize:500` + `auditLog.findMany take:5000` at `:336-341`)	500 requests + 5000 audit rows to resolve `logged_by`	client slice
`getAllUnifiedAssets` (`personnel/assets/actions.ts:114-118,350-356,673-696`)	`asset.findMany take:500` + `inventoryItem.findMany take:500`, merged ~1000 rows, per-row `await generateAssetQrDataUrl()` (`:165-206`), JS `localeCompare` sort	`assets/page.tsx:185-188 slice`
`getAllLogs`	`take:500 ORDER BY created_at DESC`	client slice

Consequences: large TTFB + large Server Action payload + main-thread filter (`assets/page.tsx:137-183` builds a 20-field haystack per row; `distinctValues` loops all rows). This cost is paid **twice** per mutation: once by the action's `bust + revalidate`, once by the client's `refresh()` refetch.

A4. Missing indexes → sequential scans [MEDIUM]

`prisma/schema.prisma` (as-found): `profiles` has no `@@index` on `role/status`; `deliveries` none on `created_at/received_by/inspection_status`; `inspections` none on `delivery_id/inspector_id`; `requests` only `@@index([recipient_id])` while filters use `status/request_type/date_requested` +

`contains(mode:insensitive)` ; `audit_logs` no index while queries do `ORDER BY created_at DESC take:500/5000` ; `inventory` no index on `quantity/reorder_threshold` scan in `dashboard/stats.ts:96-122` .

A5. Dashboard blocks on two full-scan aggregations [MEDIUM]

`dashboard/page.tsx:50-56` awaits `getSuperAdminSession()` + `getSuperAdminOverview()` before first paint. `stats.ts:64` (`withScopedCache 60s`): 8-way `Promise.all` + second 5-way `Promise.all` + `inventoryItem.findMany({ select: { quantity, reorder_threshold } })` full download with JS loop tally (`:114-122`) + N+1 `profile.findMany({ where: { id: { in: ids } } })` (`:143-145`). `analytics.ts:87` (60s): 8-way `Promise.all` with `groupBy` + full `inventoryItem.findMany` scan again. No `revalidate` export; relies on Redis TTL.

---

4. Finding B — Why "data is not instant without refresh"

B1. Double TTL stacking = up to ~2 min stale by design [HIGH]

- Server: `withScopedCache('super-admin:deliveries-list', 30)` , `('super-admin:requests-list', 30)` , `('super-admin:overview', 60)` , `('super-admin:analytics', 60)` .
- Client: `useCachedAction(KEY, fetcher, { staleTime: 30_000|60_000 })` (`requests/page.tsx:39-43` , `assets/page.tsx:88-99`), `maxAge: 10min` + `sessionStorage` persistence (`use-cached-action.ts:54,85-99`).

  Fresh client cache → **skip network entirely** (`:91-93`). Stale → silent background revalidate. `layout.tsx:8 force-dynamic` does not bypass either layer.

B2. Realtime wired on 3 pages only, fails silently [HIGH]

`src/hooks/use-admin-realtime.ts:31-93` : one channel per page, `postgres_changes` per table, events coalesced to min 2s (`:53-66`), `catch { return }` (`:79-82`) silently no-ops if RLS/publication blocks. Inline comment claims "SWR polling remains the fallback" — **no poller exists**.

Wired:

- `users/page.tsx:62-66` (`profile`), `requests/page.tsx:62-66` (`request`), `repairs/page.tsx:40-44` (`repair`)

  Missing (require manual refresh / TTL expiry):

- `assets/page.tsx` , `deliveries/page.tsx` , `inventory/page.tsx` , `inspections/page.tsx` , `issuances/page.tsx` , `documents/page.tsx` , `issues/page.tsx` , `logs/page.tsx` , `records/page.tsx` , `transactions/page.tsx` (`:59-76` fires 5-way `Promise.all` with no `refresh` exposed), `master-data/page.tsx` , `dashboard/page.tsx` (pure Server, no subscription).

B3. `revalidatePath` does not update the visible list [HIGH]

Server actions that bust + revalidate (e.g. `requests/actions.ts:79-83` , `inventory/actions.ts:63-68` , `users/actions.ts:245-248`) only invalidate Next RSC cache. The on-screen list is rendered from `useCachedAction` memory/sessionStorage (`lib/client-cache.ts:21-59,104-122`). Unless the page

also calls `refresh()` / `bustClientCache()` (which dispatches `CACHE_BUSTED_EVENT`, `use-cached-action.ts:148-157`), the user keeps seeing the pre-write payload. Only `requests/page.tsx:56-58`, `inventory ThresholdCell`, `users ActiveToggle onSuccess`, `master-forms.tsx` do this — sibling tabs and cross-role views do not.

B4. Cross-role invalidation gap [MEDIUM]

Personnel writes bust only `personnel:*`; super-admin writes bust only `super-admin:*` (`personnel-cache.ts:242-330` `bustPersonnelScopes` / `bustSuperAdminScopes`). `personnel/assets/actions.ts:578-579,664-665`, `personnel/inventory/actions.ts:313,346`, `personnel/inspections/actions.ts:866-876`, `personnel/issuances/actions.ts:927-932` revalidate only `/personnel/...` paths. So a personnel user logging a delivery/request leaves the super-admin `deliveries/requests/assets` lists stale until TTL, with no realtime subscription to close the gap (`deliveries/page.tsx` does not even destructure `refresh`).

B5. Fire-and-forget bust leaves next read stale [LOW, correctness]

`master-data/actions.ts:44-54`: `void import(...).then(bust...)` unawaited + `nextRevalidatePath`. Next read can hit stale Redis before the bust lands. A bust must either be awaited on the write path (slow but correct) or paired with an immediate client `bustClientCache` + `router.refresh()` (fast + correct) — currently neither is guaranteed.

---

5. Standard Fast Approach (recommended)

Target UX: **action reflects in <100 ms locally, server confirms in background, all viewers converge without refresh.**

5.1 Optimistic UI + silent revalidate (fixes perceived latency)

```
// requests/page.tsx — pattern to replicate on users/deliveries/inventory/master-data
const onConfirm = async () => {
  const prev = rows;
  setRows(r => r.map(x => x.id === actionRow.id ? { ...x, status: actionStatus } : x)); // instant
  setDialogOpen(false);
  try {
    const res = await setReplenishmentStatus(actionRow.id, actionStatus, note);
    if (!res.success) { setRows(prev); toast.error(...); return; } // rollback
    toast.success(successTitle);
  } catch { setRows(prev); toast.error(...); }
  finally { refresh(); } // silent background confirm, isValidating only
};
```

- Keep `loading=false` on revalidate; surface only `isValidating` dot (already supported by `BrowseTable isValidating` prop).
- Prefer `useOptimistic` or the manual patch above; always keep rollback path.

- Every mutation must call `bustClientCache(prefix)` + `refresh()`, not just `revalidatePath`.

5.2 Realtime everywhere with a real fallback (fixes staleness)

1. Add `useAdminRealtime` to every list page with exact table names: `asset`, `inventoryItem` (or `inventory_items` per schema), `delivery`, `inspection`, `issuance_records`, `request`, `repair`, `profile`, `audit_logs`.
2. Fix Supabase prerequisites: include those tables in the Realtime publication; grant `SELECT` to the super-admin role (or use a scoped broadcast channel if RLS blocks `postgres_changes`).
3. Add the missing poller fallback the hook comment promises:

   ```ts // if subscribe status !== 'SUBSCRIBED', setInterval(refresh, 15000) ```
4. Tune coalesce `2000ms → 500–800ms` for snappier convergence without thundering herd.

5.3 Server pagination + lean selects (fixes actual latency)

- Change `browseAssets / browseRequests / browseDeliveries / browseLogs` signature to `({ page, pageSize, query, status }) => ({ rows, total })` with Prisma `skip/take + where + orderBy`. Page size 25–50, not 500.
- Remove per-row QR generation from list queries; generate QR lazily when the QR dialog opens.
- Replace `auditLog take:5000` scan with a page-scoped lookup (only IDs on the current page) or a `profile` relation join.
- Replace dashboard JS tallies with SQL `COUNT WHERE` (personnel `dashboard/actions.ts:131-136` already shows the pattern).

5.4 Slim the write path (fixes server latency)

- `await` only what the response needs: auth → validate → single `prisma.$transaction([...write, auditLog...])`.
- Move `bustSuperAdminScopes` (Redis SCAN/DEL) and non-critical side effects out of the critical path via `after()` (Next `server-only`) or a job queue; pair with immediate client optimistic patch so ordering stays correct.
- Collapse N+1 profile lookups into one `findMany({ where: { id: { in: pageIds } } })` per page (already done per-page in browse readers — keep it, do not regress).
- Fix `master-data/actions.ts` fire-and-forget: either `await` the bust or await nothing server-side but `bustClientCache` + `refresh` client-side immediately.

5.5 Indexes (one migration, measurable win)

```
@@index([status, date_requested])      // requests list + filters
@@index([created_at(sort: Desc)])      // deliveries / audit_logs ordering
@@index([inspection_status])           // deliveries filter
@@index([delivery_id])                 // inspections join
@@index([role])                        // profiles role filter
@@index([action, created_at])          // audit_logs scan
```
```

Replace `contains(mode: insensitive)` full-scan search with prefix/trigram search or server-filtered pagination.

5.6 Dashboard streaming

Keep `dashboard/page.tsx` as Server Component but ensure `overview` and `analytics` each sit in their own `<Suspense fallback={skeleton}>` (partially done in `secondary.tsx`) so one slow aggregation never blocks the other or the shell.

---

6. Prioritized Action Plan

Priority	Item	Files	Effort	Effect
P0	Optimistic patch + `refresh()` on requests + users toggles	`requests/page.tsx`, `users/page.tsx`, `user-actions.tsx`	S	Spinner → instant
P0	Add `useAdminRealtime` + 15s poll fallback to deliveries, assets, inventory, inspections, issuances, logs	`deliveries/page.tsx`, `assets/page.tsx`, + hook	S–M	No manual refresh
P1	Server pagination for `browseDeliveries` / `browseRequests` / `browseAssets`; drop `take:500/5000`	`browse/actions.ts`, personnel list actions, `BrowseTable`	M	TTFB ↓, payload ↓
P1	Defer QR generation out of list path	`personnel/assets/actions.ts`, `RequestQrButton`	S	CPU ↓
P1	Slim write path: `$transaction`, `after()` for audit+bust	`requests/actions.ts`, `users/actions.ts`, `dashboard/actions.ts`	M	Action ms ↓
P2	Prisma indexes migration + replace insensitive `contains`	`prisma/schema.prisma`	S	DB scan ↓
P2	Dashboard SQL counts + Suspense split	`dashboard/stats.ts`, `analytics.ts`, `page.tsx`	S–M	FCP ↓
P2	Unified bust map: personnel writes also bust `super-admin:*` (narrow) + always `bustClientCache`	`personnel-cache.ts`, `client-cache.ts`, all mutation call sites	M	Cross-role stale gone

Acceptance criteria:

- ☐ Approve/Reject/Deactivate reflects in UI in <200 ms (optimistic), success toast follows server confirm.
- ☐ Two browsers open on the same list converge without refresh (realtime or ≤15s poll).

- ☐ No list query uses `take:500` without pagination; no per-row QR in list path.
- ☐ Writes return without awaiting Redis SCAN; audit still durable (transaction or `after()`).
- ☐ `EXPLAIN` on the five hottest list queries shows index scans.

---

7. Appendix — Evidence Index

- Pessimistic mutation: `src/app/super-admin/requests/page.tsx:92-119`, `src/app/super-admin/users/user-actions.tsx:30-47`
- Serial write path: `src/app/super-admin/requests/actions.ts:34-84`, `src/lib/personnel-cache.ts:12-46,204-223`
- Full-table reads: `src/app/super-admin/browse/actions.ts:89-93,328-341`, `src/app/personnel/assets/actions.ts:114-118,165-206,350-356,673-696`
- Client pagination: `src/components/super-admin/BrowseTable.tsx:69-71`, `src/app/super-admin/assets/page.tsx:185-188`
- Double TTL: `src/hooks/use-cached-action.ts:54,85-99`, `src/lib/client-cache.ts:21-59`, `src/app/super-admin/browse/actions.ts:87`, `src/app/super-admin/dashboard/stats.ts:64`, `analytics.ts:87`
- Realtime hook + wiring gap: `src/hooks/use-admin-realtime.ts:31-93`, `src/app/super-admin/requests/page.tsx:62-66`, `users/page.tsx:62-66`, `repairs/page.tsx:40-44` vs missing on assets/deliveries/inventory/inspections/issuances/documents/issues/logs/records/transactions/master-data/dashboard
- Cross-role bust gap: `src/lib/personnel-cache.ts:242-330`, `src/app/personnel/assets/actions.ts:578-579`, `src/app/personnel/inventory/actions.ts:313,346`, `src/app/super-admin/requests/actions.ts:79-83`
- Fire-and-forget bust: `src/app/super-admin/master-data/actions.ts:44-54`
- Heavy dashboard: `src/app/super-admin/dashboard/page.tsx:50-56`, `stats.ts:69-145`, `analytics.ts:98-126`
- Transactions page with no refresh: `src/app/super-admin/transactions/page.tsx:59-76`

  Report generated from read-only audit of `main` at `d0298a0`. No code changes made in this report commit.